

atividade a2 e p3

Otimização de árvore

Existem diversas formas de otimizar uma consulta.

- **Heurística:** aplicação de regras
- **Custo:** estimar custo
- **Semântica:** substituir uma consulta por outra

É possível gerar uma árvore de consulta, na qual as operações são executadas das folhas (entradas) até a raiz (proteção final), com os nós internos sendo as operações.

memória índice	
• Hash	
• Índice mult. estático	
• Índice mult. dinâmico	
• chave primária	→ feitos apenas sob chaves p.
• chave secundária	
• org. indexado	

exemplo passo a passo

```
SELECT a.cpf, b.nome
FROM funcionarios a, clientes b, aluguel c, funcionarios d
WHERE a.cpf = b.cpf
AND b.cpf = c.cpf_cliente
AND b.sexo = "M"
AND c.valorPagor > 50
AND a.cpf = d.cpf_supervisor
```

Agora, vamos tentar montar as árvores?

Passo 01

Analisar índices para ver quais op fazemos primeiro

No enunciado fala que todos as chaves est. tem IS

- 01 → chave p. com chave p
- 02 → chave p. com chave p.
- 03 → chave p. com chave ex.

Passo 02

Identificar quais índices poderiam ser usados seguindo a ordem de otimização

- 01 → IME - IME
- 02 → IME - IME
- 03 → IME - IS

→ pior, fica por último

Passo 03

Determinar os tamanhos das tabelas baseado no número de registros.

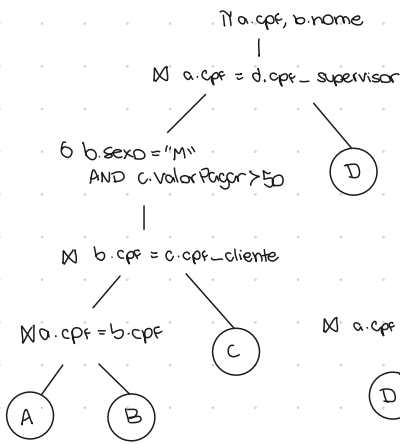
funcionários → 3.500

aluguel → 20.000.000

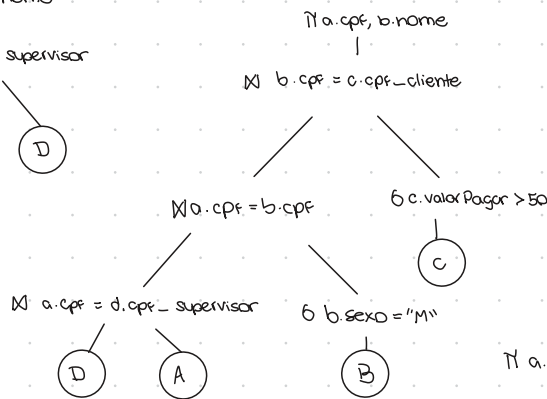
clientes → 100.000

A ideia é varrer o menor e pesquisar no maior

Árvore inicial (Baseada na ordem do from)

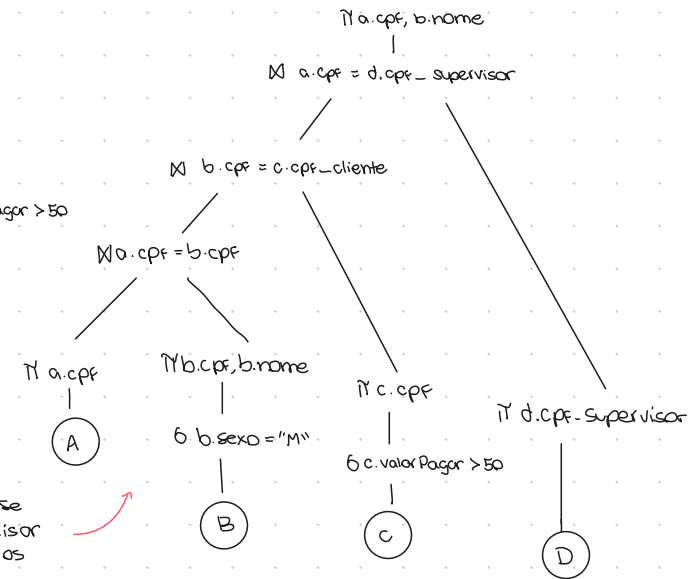


Árvore reversa



Obs: Na árvore otimizada, começamos selecionando se tiver. Depois, proteções APENAS o que vamos precisar para fazer os joins e proteções. Por fim, partimos para os joins de acordo com os índices e tamanhos.

Árvore otimizada



Agora, vamos tentar reescrever as consultas (reversa e otimizada) e elaborar um plano de execução baseado nos índices e tamanhos dos arquivos

Consulta reversa reescrita

```
SELECT a.cpf, b.nome
FROM funcionarios d, aluguel c, clientes b, funcionarios a
WHERE a.cpf = d.cpf_supervisor
AND b.sexo = "M"
AND a.cpf = b.cpf
AND c.valorPagor > 50
AND b.cpf = c.cpf_cliente
```

Consulta otimizada reescrita

```
SELECT a.cpf, b.nome
FROM funcionarios a, clientes b, aluguel c, funcionarios d
WHERE b.sexo = "M"
AND a.cpf = b.cpf
AND c.valorPagor > 50
AND b.cpf = c.cpf_cliente
AND a.cpf = d.cpf_supervisor
```

Plano de execução

Serve para saber quais algoritmos e índices serão usados em cada etapa da sua consulta. Além disso, podemos calcular o custo e a complexidade, o que é útil para diagnosticar e modificar a consulta.

01. Filtrar clientes por sexo → pesquisa linear
02. Junção de loop único → varrer a e pesquisar em b
03. Filtrar valor a pagar → pesquisa linear
04. Junção de loop único → varrer ab e pesquisar em c
05. Junção de loop único → varrer abc e pesquisar em d

Pesquisa em índice multiúvel estático

Pesquisa em índice secundário

dos: Se tiver índice dos dois lados, eles serão usados, o não ser que tivesse algum problema ou um dos dois estivesse corrompido

Otimização de consulta

Podemos fazer algo denominada **sintaxia de consulta** ou tuning → avaliar req. físicos de um projeto para reescrever consultas

- Ordem aleatória de tabelas no FROM ou ops no WHILE
- Comparações usando NULL, substrings ou campos de domínios diferentes
- Operadores SOME, IN, ALL, ANY, EXISTS, DISTINCT
- Operador OR ou UNION
- Subconsultas (select dentro de select)

→ comparar dois casos diferentes, como salário e número

Transações

São conjuntos de operações de acesso a um BD

Alguns conceitos:

- **Concorrência**: múltiplas transações ao mesmo tempo
- **Independência**: uma transação não afetar na execução da outra
- **Granularidade**: tamanho dos dados.
 - ↳ + fina: maior concorrência, maior controle (ex: adunas da tabela)
 - ↳ + grossa: menor concorrência, menor controle (ex: blocos)

Para transações concorrentes, pode-se ocorrer:

Atualização perdida

transações escrevem o mesmo item e uma delas sobrescreve a outra

Leitura suja

transação atualiza um item e depois, e outra transação lê o item sujo

Leitura não repetitiva

a mesma transação lê em dois momentos diferentes um item, tendo o seu valor sido mudado

resumo incorreto

transação faz agregação com valores incorretos pois foram atualizados por outras

Uma execução concorrente de transações pode estar sujeita a vários falhas, como:

- **Sistema**: hardware, software ou rede
- **Operação**: programação ou usuário
- **Concorrência**: técnicas de controle
- **Condição de exceção**: programada na transação
- **Disco**: blocos de dados perdidos
- **Catástrofe**: blackout, incêndio, raios

SGBDs lidam com:

Logging

histórico das ts, usado quando sistema cai

Dumping

cópia de segurança no comp. usado quando o comp. cai

Backup

cópia de segurança externa usada quando servidor morre

Operações utilizadas

- b → begin
- e → end
- c → commit
- a → abort
- w → write
- r → read

Uma boa transação deve garantir as propriedades ACID:

1. Atomicidade

Uma T é uma unidade atômica, ou seja, ou executa integralmente ou não executa

2. Isolamento

Execução independente de outras transações

3. Durabilidade

Transações confirmadas devem persistir

4. Consistência

Restrições especificadas devem ser respeitadas a todo momento

Para garantir transações que cumpram as propriedades ACID, é feito um escalonamento (intercalação).

Esses escalonamentos também possuem propriedades

propriedades de um escalonamento

Para determinar os ciclos de um escalonamento, é feito um grafo de precedência a partir dos conflitos

Conflitos

acontecem quando:

- transações diferentes
- operam sobre o mesmo item
- pelo menos uma é write

Grafo de precedência

Quando há conflito, transação que vem depois aponta para a que precede

$w_1(x), w_2(x)$



Serializável

escalonamentos não serializáveis (que tenham concorrência) mas se comportam como serializáveis (sem interferência)

se tem ciclo geral, não é serializável

recuperável

transação que está lendo confirma antes da que escreveu

completo

- todas as ops. originais presentes
- ordem das ops. preservada
- sem ciclos entre duas ou mais operações

Controle de concorrência

Para garantir que as transações ocorram ao mesmo tempo mas que uma não atrapalhe a outra, existem técnicas de controle

das: o que o protocolo gera ou é serial ou é serializável

Considere que o escalonamento S_a apresentado abaixo foi constituído a partir das transações T_1 , T_2 e T_3 também apresentadas abaixo. Ressalta-se que, em um SGBDR diversas transações devem ser escalonadas para executarem simultaneamente, aumentando assim a concorrência e, consequentemente, diminuindo o tempo de processamento. No entanto, tal concorrência demanda a utilização de técnicas de controle de concorrência para garantir as propriedades de Atomicidade, Consistência, Isolamento e Durabilidade (ACID).

$$T_1 = r(x), r(y), w(x), r(z)$$

$$T_2 = r(z), r(x), r(y), w(z)$$

$$T_3 = r(y), r(z), w(y), r(x)$$

$$S_a = r_3(y), r_2(z), r_1(x), r_2(x), r_3(z), r_2(y), w_3(y), r_1(y), w_2(z), w_1(x), r_3(x), r_1(z)$$

protocolo de bloqueio binário

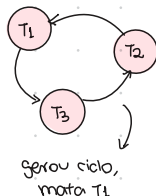
- Bloqueio, desbloqueio
- Se está bloqueado, outro não pode ler ou escrever

$$Pb \rightarrow l_3(y), l_2(z), l_1(x), l_2(x), l_3(z), l_3(x)$$

$$Fe \rightarrow r_2(x), r_3(z), r_2(y), w_3(y), r_1(y)$$

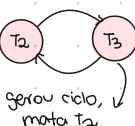
$$Sa \rightarrow r_3(y), r_2(z), r_1(x), r_2(x), r_3(z), w_3(y), r_3(x)$$

- T_2 espera T_1
- T_3 espera T_2
- T_1 espera T_3



gerou ciclo, mata T_1

- T_3 espera T_2
- T_2 espera T_3



gerou ciclo, mata T_2

protocolo de bloqueio read-write

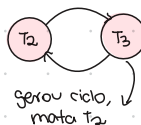
- Bloqueio read, write e desbloqueio
- Se está bloqueado para escrita, não faz nada
- Se está bloqueado para leitura, outras leem

$$Pb \rightarrow r_3(y), l_2(z), l_1(x), r_2(x), r_3(z), r_2(y), r_1(y), w_3(y), w_1(x), r_3(x)$$

$$Fe \rightarrow w_3(y), w_2(z)$$

$$Sa \rightarrow r_3(y), r_2(z), r_1(x), r_2(x), r_3(z), r_2(y), r_1(y), w_1(x), r_1(z), w_3(y), r_3(x)$$

- T_3 espera T_2
- T_2 espera T_3



gerou ciclo, mata T_2

T_3 espera T_1

pros and cons

É importante ressaltar que o SGBD não usará apenas uma técnica. Ele usará várias, quando for necessário.

Cada uma tem suas vantagens e desvantagens.

O protocolo de bloqueio ternário, por exemplo, gera mais concorrência entre as transações, mas não é imune aos deadlocks. Já o protocolo de timestamp é imune a deadlocks, mas diminui a concorrência e mata mais do que realmente precisa.

protocolo de timestamp

Considere que o escalonamento S_a apresentado abaixo foi constituído a partir das transações T_1 , T_2 e T_3 também apresentadas abaixo. Ressalta-se que, em um SGBDR diversas transações devem ser escalonadas para executarem simultaneamente, aumentando assim a concorrência e, consequentemente, diminuindo o tempo de processamento. No entanto, tal concorrência demanda a utilização de técnicas de controle de concorrência para garantir as propriedades de Atomicidade, Consistência, Isolamento e Durabilidade (ACID).

$$T_1 = r(x), r(y), w(x), r(z)$$

$$T_2 = r(z), r(x), r(y), w(z)$$

$$T_3 = r(y), r(z), w(y), r(x)$$

$$S_a = r_3(y), r_2(z), r_1(x), r_2(x), r_3(z), r_2(y), w_3(y), r_1(y), w_2(z), w_1(x), r_3(x), r_1(z)$$

Para resolução do protocolo de timestamp, é feita uma tabelinha:

ts	rts	wts
itens		
x	0	0
y	0	0
z	0	0

As regras são:

- leitura com leitura não há problema
- transações mais antigas não podem manipular itens já manipulados por transações mais novas

Definir a idade

$T_1 \rightarrow 3 \rightarrow$ nasceu por último

$T_2 \rightarrow 2$

$T_3 \rightarrow 1 \rightarrow$ nasceu primeiro

- Tabelinha começa com 0

$$S_a = r_3(y), r_2(z), r_1(x), r_2(x), r_3(z), r_2(y), r_1(y), w_2(z), w_1(x), r_1(z)$$

passo a passo:

$r_3(y) \rightarrow$ leitura com leitura
 $0 > 1$? não! Logo, posso ler

$r_2(z) \rightarrow$ leitura com leitura
 $0 > 2$? não! Logo, posso ler

$r_1(x) \rightarrow$ leitura com leitura
 $0 > 3$? não! Logo, posso ler

$r_2(x) \rightarrow$ leitura com leitura
 $3 > 2$? sim, posso ler mas não substitui

$r_3(z) \rightarrow$ leitura com leitura
 $2 > 1$? sim, posso ler mas não substitui

$r_2(y) \rightarrow$ leitura com leitura
 $1 > 2$? não! Logo, posso ler

0	0
1	0
0	0
0	0
0	0
1	0
2	0
3	0
1	0
2	0
3	0
1	0
2	0
3	0
1	0
2	0
3	0
2	0
2	0

$w_3(y) \rightarrow$ escrita analisa as duas
 $2 > 1$? Sim, Logo, não pode escrever
 T_3 é abortada

$r_1(y) \rightarrow$ leitura com leitura
 $2 > 3$? não, logo, posso ler

$w_2(z) \rightarrow$ escrita analisa as duas
 $2 > 2$? não $0 > 2$? não
 logo, posso escrever

$w_1(x) \rightarrow$ escrita analisa as duas
 $3 > 3$? não $0 > 3$? não
 logo, posso escrever

$r_1(z) \rightarrow$ leitura com leitura
 $2 > 3$? não, logo, posso escrever

3	0
3	0
2	0
3	0
3	0
2	2
3	3
3	0
2	2
3	3
3	0
3	2